

ORCS 4529: Reinforcement Learning

Shipra Agrawal

Columbia University
Industrial Engineering and Operations Research

Contents I

MDP

Dynamic Programming (DP) based algorithms for RL

Policy Gradient Methods

Monte Carlo Tree Search (MCTS)

MCTS

Contents I

MDP

Dynamic Programming (DP) based algorithms for RL

Policy Gradient Methods

Monte Carlo Tree Search (MCTS)

MCTS

Contents I

MDP

Dynamic Programming (DP) based algorithms for RL

Policy Gradient Methods

Monte Carlo Tree Search (MCTS)

MCTS

Contents I

MDP

Dynamic Programming (DP) based algorithms for RL

Policy Gradient Methods

Monte Carlo Tree Search (MCTS)

MCTS

Monte Carlo Tree Search (MCTS)

- ▶ Bandit based Monte-Carlo Planning Koscis and Szepesvari 2006
- ▶ From Bandits to Monte-Carlo Tree Search: The Optimistic Principle Applied to Optimization and Planning, Foundations and Trends in Machine Learning, Vol. 7, No. 1 (2014) 1–129, 2014 Remi Munos.

Historical Motivation for MCTS

- ▶ Simulation-based approach to game programming

Historical Motivation for MCTS

- ▶ Simulation-based approach to game programming
 - ▶ game simulators that allow state resets: can run many playouts from any “position”.

Historical Motivation for MCTS

- ▶ Simulation-based approach to game programming
 - ▶ game simulators that allow state resets: can run many playouts from any “position”.
 - ▶ reward typically only at the end of a playout.

Historical Motivation for MCTS

- ▶ Simulation-based approach to game programming
 - ▶ game simulators that allow state resets: can run many playouts from any “position”.
 - ▶ reward typically only at the end of a playout.
 - ▶ branching factor is so large that regular minimax and alpha-beta search methods fail

Historical Motivation for MCTS

- ▶ Simulation-based approach to game programming
 - ▶ game simulators that allow state resets: can run many playouts from any “position”.
 - ▶ reward typically only at the end of a playout.
 - ▶ branching factor is so large that regular minimax and alpha-beta search methods fail
- ▶ New approach to playing Computer Go [Coulom 2006]:

Historical Motivation for MCTS

- ▶ Simulation-based approach to game programming
 - ▶ game simulators that allow state resets: can run many playouts from any “position”.
 - ▶ reward typically only at the end of a playout.
 - ▶ branching factor is so large that regular minimax and alpha-beta search methods fail
- ▶ New approach to playing Computer Go [Coulom 2006]:
 - ▶ Assign more weight to moves with better scores in previous runs

Historical Motivation for MCTS

- ▶ Simulation-based approach to game programming
 - ▶ game simulators that allow state resets: can run many playouts from any “position”.
 - ▶ reward typically only at the end of a playout.
 - ▶ branching factor is so large that regular minimax and alpha-beta search methods fail
- ▶ New approach to playing Computer Go [Coulom 2006]:
 - ▶ Assign more weight to moves with better scores in previous runs
 - ▶ Adaptively explore most promising branches to a greater depth

Historical Motivation for MCTS

- ▶ Simulation-based approach to game programming
 - ▶ game simulators that allow state resets: can run many playouts from any “position”.
 - ▶ reward typically only at the end of a playout.
 - ▶ branching factor is so large that regular minimax and alpha-beta search methods fail
- ▶ New approach to playing Computer Go [Coulom 2006]:
 - ▶ Assign more weight to moves with better scores in previous runs
 - ▶ Adaptively explore most promising branches to a greater depth
- ▶ Tree Search + Bandits = MCTS

Historical Motivation for MCTS

- ▶ Simulation-based approach to game programming
 - ▶ game simulators that allow state resets: can run many playouts from any “position”.
 - ▶ reward typically only at the end of a playout.
 - ▶ branching factor is so large that regular minimax and alpha-beta search methods fail
- ▶ New approach to playing Computer Go [Coulom 2006]:
 - ▶ Assign more weight to moves with better scores in previous runs
 - ▶ Adaptively explore most promising branches to a greater depth
- ▶ Tree Search + Bandits = MCTS
 - ▶ Used in successful Computer Go programs such as MoGo [Gelly et al. 2006], AlphaGo [2017] and AlphaZero [2017]

Illustration of MCTS

Figure taken from [Munos 2014]

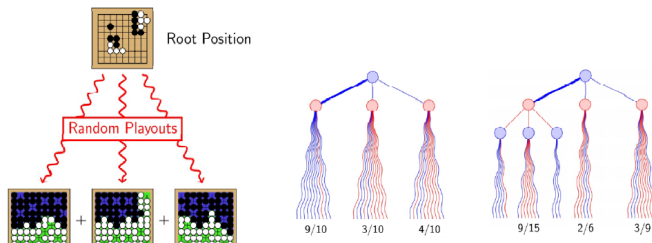


Figure 2.1: Illustration of the Monte-Carlo Tree Search approach (Courtesy of Rémi Coulom from his talk *The Monte-Carlo revolution in Go*). Left: Monte-Carlo evaluation of a position in Computer Go. Middle: each initial move is sampled several times. Right: The apparently best moves are sampled more often and the tree structure grows.

Recent Applications

Reasoning and planning with LLM agents.

Example: Writing code to implement a machine learning pipeline using LLMs

Upper Confidence bounds in Trees (UCT) algorithm

[Kocsis and Szepesvári, 2006]

- ▶ MCTS: Evaluate from actual performance in rollouts instead of hand-crafted evaluation or heuristic scoring functions
- ▶ Search with trials, but not exhaustive – focus on promising parts
- ▶ Exploration-exploitation is a major challenge

$$\text{UCB+Trees} = \text{UCT}$$

The tree search problem

- ▶ Uniform tree of depth D , each node as K children
- ▶ Each leaf node i has (apriori unknown) reward distribution ν_i , mean μ_i
- ▶ Goal: find the path to a leaf with the highest mean μ_i
- ▶ Challenge: Search over K^D leaves and paths

UCT algorithm

For each round t

- ▶ Select a leaf node l_t by taking the maximum score path from root to leaf.
- ▶ Receive a reward $r_{l_t} \sim \nu_{l_t}$.
- ▶ Use the reward to update the scores of all nodes in the path

Prioritizes most promising branches according to previously observed sample rewards

UCT algorithm

For each round t

- ▶ Select a leaf node l_t by taking the maximum score path from root to leaf.
 - ▶ Starting from the root, at each node j , select the child k with the highest score $B_t(k)$, $k = 1, \dots, K$, where

$$B_t(k) := \hat{\mu}_{k,t} + c \sqrt{\frac{\log(n_{j,t})}{n_{k,t}}}$$

where $n_{j,t}$, $n_{k,t}$ are the number of visits to nodes j , k resp. and

$$\hat{\mu}_{k,t} := \frac{1}{n_{k,t}} \sum_{s=1}^t r_s \mathbb{I}(l_s \in \mathcal{L}(k))$$

$\mathcal{L}(k)$ is the set of leaves in the subtree rooted at node k .

- ▶ Receive a reward $r_{l_t} \sim \nu_{l_t}$.
- ▶ Use the reward to update the scores of all nodes in the path
 - ▶ Update $\hat{\mu}_{j,t}$ and $B_t(j)$ for all j in $\text{Path}(\text{root} \rightarrow l_t)$.

Theoretical guarantees

Consistency

- ▶ Easy to see that in an infinite number of simulations, each node will be selected infinitely often so that for all leaf nodes i

$$\hat{\mu}_{i,t} \rightarrow \mu_i$$

and for all intermediate nodes j , we have

$$\hat{\mu}_{j,t} \rightarrow \max_{i \in \mathcal{L}(j)} \mu_i, \quad B_{j,t} \rightarrow \max_{i \in \mathcal{L}(j)} \mu_i$$

- ▶ So eventually the path to the leaf node with the highest μ_i will be selected.

Theoretical guarantees

Finite time performance

Can we apply regular UCB analysis for the N-armed bandit problem?

- ▶ Consider the problem of each node trying to select one of its children $k = 1, \dots, K$, as a K -armed bandit problem.
- ▶ Value of k th child: $V(k) = \max_{i \in \mathcal{L}(k)} \mu_i$.
- ▶ Optimal arm: $\arg \max_k V(k)$
- ▶ empirical mean for k is not an unbiased estimate for its value

$$\mathbb{E}[n_{k,t} \hat{\mu}_{k,t}] = \mathbb{E} \left[\sum_{s=1}^t r_s \mathbb{I}(I_s \in \mathcal{L}(k)) \right] = \sum_{i \in \mathcal{L}(k)} n_{i,t} \mu_i$$

Gets closer to $\max_{i \in \mathcal{L}(k)} \mu_i$ if leaves with large μ_i rare chosen more often.

- ▶ $B(k) = \hat{\mu}_{k,t} + c \sqrt{\frac{\log(n_{j,t})}{n_{k,t}}}$ not an optimistic estimate of $V(k)$.

UCB can handle slight drift

Consider the bandit problem where distribution of arm i is non-stationary, with

$$\mathbb{E}[\hat{\mu}_{i,n}] = \mu_i + \delta_{i,n}$$

But, for large enough $n \geq N_0$, the drift is bounded as

$$\delta_{i,n} \leq C \sqrt{\frac{\ln s}{s}}$$

and as $n \rightarrow \infty$, $\mu_{i,n} \rightarrow \mu_i$

Theorem (Auer et al 2002)

Consider UCB (with constant $2C$ multiplied to the bonus term) applied to a non-stationary problem. Let $n_{i,T}$ denote the total number of plays of arm i . Then for any suboptimal arm i ,

$$\mathbb{E}[n_{i,T}] \leq 64C^2 \frac{\ln n}{\Delta_i^2} + 2N_0 + \frac{\pi^2}{3}$$

Analysis of UCT

What is connection to the non-stationary bandit?

- ▶ For leaf node, we have a stationary bandit problem

Analysis of UCT

What is connection to the non-stationary bandit?

- ▶ For leaf node, we have a stationary bandit problem
- ▶ For intermediate node, $\mathbb{E}[\hat{\mu}_{j,t}]$ for node j is getting closer and closer to $\mathbb{E}[\max_{i \in \mathcal{L}(j)} \mu_i]$. The drift is smaller with larger number of plays.

Analysis of UCT

What is connection to the non-stationary bandit?

- ▶ For leaf node, we have a stationary bandit problem
- ▶ For intermediate node, $\mathbb{E}[\hat{\mu}_{j,t}]$ for node j is getting closer and closer to $\mathbb{E}[\max_{i \in \mathcal{L}(j)} \mu_i]$. The drift is smaller with larger number of plays.
- ▶ Using induction we can bound the drift as required for large enough N_0 . However, the "constant" N_0 may depend on the tree depth.

Analysis of UCT

What is connection to the non-stationary bandit?

- ▶ For leaf node, we have a stationary bandit problem
- ▶ For intermediate node, $\mathbb{E}[\hat{\mu}_{j,t}]$ for node j is getting closer and closer to $\mathbb{E}[\max_{i \in \mathcal{L}(j)} \mu_i]$. The drift is smaller with larger number of plays.
- ▶ Using induction we can bound the drift as required for large enough N_0 . However, the "constant" N_0 may depend on the tree depth.
- ▶ Can use the non-stationary bandit result to produce $O(K \log(T))$ asymptotic regret bound for UCT

Analysis of UCT

What is connection to the non-stationary bandit?

- ▶ For leaf node, we have a stationary bandit problem
- ▶ For intermediate node, $\mathbb{E}[\hat{\mu}_{j,t}]$ for node j is getting closer and closer to $\mathbb{E}[\max_{i \in \mathcal{L}(j)} \mu_i]$. The drift is smaller with larger number of plays.
- ▶ Using induction we can bound the drift as required for large enough N_0 . However, the "constant" N_0 may depend on the tree depth.
- ▶ Can use the non-stationary bandit result to produce $O(K \log(T))$ asymptotic regret bound for UCT
- ▶ The additive "constants" in regret may be arbitrarily bad, so finite time performance may suffer.

Finite-time performance: Bad example

Figure taken from Munos 2014

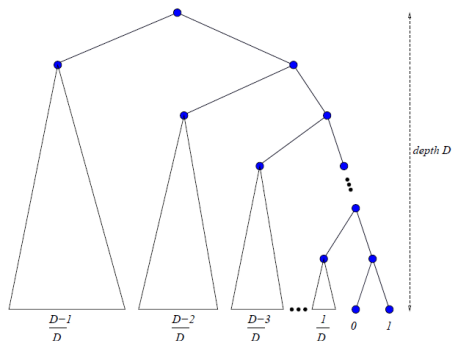


Figure 2.2: An example of tree for which UCT performs very poorly.

$$\text{Regret of UCT is } c \cdot \underbrace{\exp(\exp(\cdots \exp(1)) \cdots)}_{(D \text{ times})} + \Omega(\log(T)).$$

ORCS 4529: Reinforcement Learning

└ Monte Carlo Tree Search (MCTS)

└ MCTS

└ Finite-time performance: Bad example

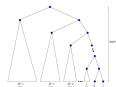
Finite-time performance: Bad example
Figure taken from Munos 2014

Figure 2.8: An example of tree for which MCTS performs very poorly.

Regret of UCT is $c \cdot \underbrace{\exp(\exp(\dots \exp(1)) \dots)}_{(D \text{ times})} + \Omega(\log(T))$

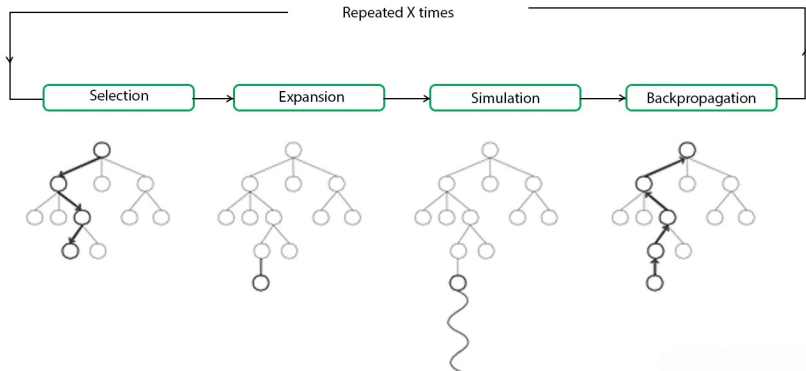
In this example, initially until the right branch is explored enough, average reward from left branches is more, making it played more by the algorithm.

Specifically, (text taken from Munos 2014) as long as the optimal reward has not been observed, from any node along the optimal path, the left branches seem better than the right ones and are thus explored exponentially more often (since out of n samples, UCB pulls only $O(\log n)$ times sub-optimal arms, as seen previously). Therefore, the time required before the optimal leaf is eventually reached is huge and we can deduce the following lower-bound on the regret of UCT: $Reg_n = c \cdot \underbrace{\exp(\exp(\dots \exp(1)) \dots)}_{(D \text{ times})} + \Omega(\log(n))$

The first term of this bound is a constant independent of n (thus the regret is asymptotically of order $\log n$ as proven in [Kocsis and Szepesvári, 2006]) but this constant is “D-uply” exponential. In particular this is much worse than a uniform sampling of all the leaves which will be “only” exponential in D .

General framework of MCTS

Scaling to large state-spaces/large trees



Gymnasium based Code for Cartpole : https://medium.com/@_michelangelo_/monte-carlo-tree-search-mcts-algorithm-for-dummies-74b2bae53bfa

Uses "deepcopy" function for taking multiple actions to create children

MCTS algorithm

specific implementations may differ

- ▶ **Selection:** Starting from the root of the current tree, from each node select the child with the highest "Score". Stop when a leaf node is reached.
- ▶ **Expansion:** If the leaf node has been visited before and has children, create child(ren) of this node and add them to the tree – one for each action. (Use the function **deepcopy** in gymnasium). Pick a random child and make it the current node.
- ▶ **Simulation:** Do a random rollout from the current node (game state) according to the current policy (e.g., arg max of scores), until the episode finishes. Increase the number of visits of the current node by 1.
- ▶ **Backpropagation:** use the reward at the end of rollout to update the "score" of parent nodes in the trajectory.

Scores in MCTS

- ▶ UCT: UCB based scores, updated only when a node (or state corresponding to that node) is visited
- ▶ Alphazero: rewards are used to update a value network ($V(s)$) and policy network ($\pi(s, a)$ or $Q(s, a)$). Score is a combination of UCT score, value score and policy score.
 - ▶ Scores updated for similar states
 - ▶ Partial lookaheads with bootstrapping/TD-learning or full lookahead Monte-Carlo method may be used to update value function
- ▶ MuZero:
 - ▶ Learns dynamics of environment to simulate a rollout faster
 - ▶ Tracks upper and lower bounds on the value of a node. Uses it to normalize the score.